

SE Lab – 3

Book Club Reading Challenge & Discussion Portal

Written Justification

Architecture Selection

The system is split into a Presentation layer (Web UI Component), a Business layer (Authentication & Access Control, Discussion Manager, Reading Progress & Goal Manager, Poll & Voting Manager) and a Data layer (Data Access Component). Components talk only through provided/required interfaces (IAuth, IDiscussion, IReading, IVoting, IDataAccess).

Why Not the Alternatives

Style	Fit for this portal
Layered	Selected. Matches the portal's shape: UI, then feature logic, then data. Simple to build, test and explain.
Microservices	Rejected. A club portal has low traffic, so independent scaling is not needed. Network latency, deployment overhead and distributed data would make vote counting harder for no benefit.
Client-Server	Valid but too coarse. It says where code runs, not how the server side is organised into modules, so it would not give the component structure this lab asks for.

Reasons for This Choice

1. Clear separation of concerns for the portal's features.

Each requirement maps to one business component: spoiler hiding and chapter discussions (FR-001, FR-004) to the Discussion Manager, reading goals and page updates (FR-002, FR-003) to the Reading Progress & Goal Manager, and monthly voting (FR-005) to the Poll & Voting Manager. The UI can change without touching this logic, which supports the ease-of-use goal (NFR-002).

2. One central data layer keeps votes consistent.

NFR-001 requires one vote per member and up-to-date poll results. Because every business component reaches data only through IDataAccess, the duplicate-vote check and the vote write happen in one place against one database, which avoids the data-consistency problems that distributed services would introduce.

Security Advantage

The UI never accesses the database directly. Every request passes through the Authentication & Access Control component (IAuth) and the business layer before reaching data. This lets the system enforce roles in one place: only Administrators manage polls and challenges, only Discussion Leads manage threads, and members cannot see spoiler content unless they reveal it. Using only IDataAccess for storage also limits exposure to injection attacks and unauthorised data access.

Performance Benefit

Layers are called in-process (direct method or function calls) rather than over the network, so a request such as submitting a vote has no inter-service latency. The Data Access Component can also cache frequently read data such as poll options and results, so refreshing poll results after each vote stays fast. If load grows, the whole application can be scaled horizontally behind a load balancer. Trade-off: layers are tightly coupled to their neighbours and cannot be scaled individually. This is acceptable at book-club scale.